

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
“JNANA SANGAMA”, BELAGAVI –590 018.



2026 – 2027

**MINI PROJECT ON
DEEP LEARNING (BCA701)**

“Real-Time Recurrent Learning for Aircraft Surface Controllers”

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

CSE (AI&ML)

Submitted By

Nipun Joisa [1RN23CI100]

Under the guidance of

Ms . JAYASHREE NAGARAJ

Assistant Professor, Dept. of CSE (AI&ML)

DEPARTMENT OF CSE (AI&ML)



Estd : 2001

RNS INSTITUTE OF TECHNOLOGY

*Autonomous Institution Affiliated to VTU, Recognized by GOK, Approved by AICTE
RajaRajeshwari Nagar, Dr. Vishnuvardhan Road, Bengaluru – 560 098*

ACKNOWLEDGEMENT

At the very onset, I would like to place my gratefulness to all those people who helped me in making this project a successful one.

I would like to thank the Management of RNSIT for providing a healthy environment for the successful completion of project work.

I would like to express my thanks to **Dr. M K Venkatesha**, Director, RNSIT for his constant support and motivating me towards the attainment of knowledge.

I express my sincere gratitude to **Dr. Ramesh Babu H S**, Principal, RNSIT for providing me with all the facilities in this college.

I am grateful to **Dr. Andhe Pallavi**, Professor and Head of the Department of Artificial Intelligence & Machine Learning for her support and encouragement.

I thank my **Ms. JAYASHREE NAGARAJ** Assistant Professor, Department of Artificial Intelligence & Machine Learning for having guided the project.

I also thank all the Teaching and Non-Teaching staff members of our department for helping me out at all times.

I thank my beloved friends for having supported me with all their strength and might. Last but not the least; I thank my parents for supporting and encouraging me throughout. I made an honest effort in this assignment.

Nipun Joisa - 1RN23CI100

TABLE OF CONTENTS

Content	Page No
Abstract	1
Introduction	1
Objectives	1
Dataset Description	1-2
Tools and Technologies Used	2
Methodology	2-3
System / Pipeline Architecture	4
Results	4-7
Conclusion	7

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1	End-to-end pipeline, from the JSBSim simulator to the metrics and figures reported here.	4
2	Nominal scenario — attitude tracking RMSE by controller.	5
3	Wind scenario — attitude tracking RMSE under Dryden turbulence.	5
4	Ablation — RTRL-RTU with online updates enabled versus disabled.	6
5	Live demo dashboard: recovery from a 50% aileron fault.	6
6	Live demo: pitch response to a 50% elevator fault.	6
7	Live demo: roll response to a 50% rudder fault.	7

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
1	Observation vector supplied to every controller (11 dimensions).	2
2	Action vector produced by every controller (3 dimensions).	2
3	Tools and technologies used.	2
4	Main results — mean $\pm$ standard deviation over three seeds.	4
5	Ablation — online updates enabled versus disabled, fault scenario.	5

1. Abstract

This project presents a deep-learning flight-surface controller for a fixed-wing aircraft that adapts its own weights during flight. A small recurrent network built from Recurrent Trace Units (RTUs) maps the aircraft's attitude state to aileron, elevator and rudder deflections, and is trained online using Real-Time Recurrent Learning (RTRL) — an exact, causal gradient method that requires no replay buffer and no unrolled backward pass through history. The elementwise RTU recurrence reduces RTRL's usual $O(N^4)$ cost to $O(N^2)$, which is what makes exact online learning tractable in real time. The controller is benchmarked in JSBSim against a tuned PID and a frozen BPTT-LSTM across four scenarios: nominal flight, mid-episode actuator faults, Dryden turbulence, and both together. An ablation that disables only the online update isolates adaptation as the causal mechanism: attitude RMSE degrades from 1.534 rad to 2.048 rad when updates are switched off, confirming that the online RTRL step, rather than the RTU architecture alone, is what drives in-flight recovery.

2. Introduction

Learned flight controllers are almost always trained offline, frozen, and then deployed. That pipeline works for as long as the airframe behaves the way it did during training, but a control surface that loses effectiveness mid-flight — through icing, damage or actuator wear — moves the aircraft outside the distribution the network ever saw, and frozen weights cannot respond to it. Real-Time Recurrent Learning offers a principled alternative: it computes exact gradients forward in time, one timestep at a time, so a network can keep learning during the very episode that damages it. RTRL was abandoned decades ago because its exact cost grows as $O(N^4)$ in the number of hidden units. Recurrent Trace Units restore tractability by making the recurrence diagonal, so each parameter's sensitivity stays local to a single unit. This project revisits in-flight adaptation with that combination, and measures what online learning actually buys over an otherwise identical frozen controller.

3. Objectives

- To implement a Recurrent Trace Unit cell and its exact forward-sensitivity recursion from first principles, and to verify the resulting gradient against finite differences and an unrolled autograd reference.
- To expose the JSBSim Cessna 172 as a Gymnasium attitude-hold environment, with composable actuator-fault and Dryden-turbulence wrappers that leak no state between episodes.
- To warm-start the recurrent controller by imitation of a tuned PID expert, and then allow Real-Time Recurrent Learning to update its weights online, one step per environment step, during deployment.
- To benchmark the RTRL controller against a tuned PID and a frozen BPTT-LSTM across nominal, fault, wind and combined scenarios, computing every metric from per-step parquet traces.
- To isolate online adaptation as the causal mechanism by means of an ablation that deploys identical warm-started weights with the online update disabled.

4. Dataset Description

Unlike a conventional supervised learning project, this controller has no fixed dataset on disk: its training data is generated by flying the simulator. The imitation warm-start uses 50 episodes of a tuned PID expert flown in calm air, each lasting 60 seconds at a 5 Hz agent rate, which yields roughly 15,000 state-action pairs. Every episode samples a fresh target pitch in $[-0.3, 0.3]$ rad and a target roll in $[-0.4, 0.4]$ rad, so the network sees

the full setpoint range rather than a single trim condition. Once deployed, the RTRL controller produces its own data as it flies — each environment step supplies exactly one online gradient. The observation and action vectors that define this data are fixed at eleven and three dimensions respectively:

Index	Property	Meaning
0	position/h-sl-ft	Altitude above mean sea level (context only)
1–2	attitude/pitch-rad, attitude/roll-rad	Current pitch and roll angles
3–5	velocities/u,v,w-fps	Body-frame velocity triple
6–8	velocities/p,q,r-rad_sec	Roll, pitch and yaw rates
9	error/pitch-error-rad	target pitch – current pitch
10	error/roll-error-rad	target roll – current roll

Table 1: Observation vector supplied to every controller (11 dimensions).

Throttle is deliberately excluded from the action space — this is an attitude-hold task, not full flight control, so throttle is held at a fixed value for the whole episode.

Index	Property	Meaning
0	fcs/aileron-cmd-norm	Aileron deflection command, normalised to $[-1, 1]$
1	fcs/elevator-cmd-norm	Elevator deflection command, normalised to $[-1, 1]$
2	fcs/rudder-cmd-norm	Rudder deflection command, normalised to $[-1, 1]$

Table 2: Action vector produced by every controller (3 dimensions).

Turbulence and actuator faults are excluded from the warm-start data by design, so the wind and fault scenarios test generalisation to disturbances the network never saw during training, rather than memorisation of them.

5. Tools and Technologies Used

Tool/Technology	Purpose
Python 3.11	Core language for the environment, controllers, training loops and analysis
PyTorch 2.4	Tensor and autograd backend for the RTU cell, the LSTM baseline and the online optimiser
jsbgym + JSBSim	Cessna 172 flight-dynamics model exposed as a Gymnasium environment
Gymnasium 1.3	Environment API, and the wrapper mechanism used for faults, wind and tracing
Hydra	Configuration management — one config per experiment, no hardcoded hyperparameters
pandas + PyArrow	Per-step parquet traces, which every metric and figure is computed from
Matplotlib	Result figures and the live interactive demo dashboard
pytest + ruff	Correctness tests (including the sensitivity check), linting and formatting

Table 3: Tools and technologies used.

6. Methodology

6.1 The Recurrent Trace Unit

The controller's core is the RTU cell defined in `rtrl/rtu_cell.py`. Its hidden state of 64 units evolves through an elementwise linear recurrence, $h_t[i] = a[i] \cdot h_{t-1}[i] + z_t[i]$, where $z_t = W_{in} \cdot x_t + b_{in}$ and the per-unit gain is parametrised as $a[i] = \tanh(a_{raw}[i])$ so that $|a[i]| < 1$ and the state can never explode regardless of how the gains are updated. All nonlinearity is pushed into the readout, $y_t = W_{out} \cdot \tanh(h_t) + b_{out}$, and never

into the recurrence itself. Because the recurrence is diagonal, $\partial h_t[i]/\partial h_{t-1}[j]$ vanishes for every $i \neq j$ — the single property that makes exact RTRL affordable here.

6.2 Forward Sensitivity and the Online Update

For each recurrent parameter θ the controller maintains a forward sensitivity $S_t[i] = \partial h_t[i]/\partial \theta$, advanced once per step by $S_t[i] = a[i] \cdot S_{t-1}[i] + x_t[k]$, or $+1$, or $+(1 - a[i]^2) \cdot h_{t-1}[i]$ for W_{in} , b_{in} and a_{raw} respectively. The instantaneous credit assignment through the readout, $\partial L_t/\partial h_t[i]$, is then combined with the stored sensitivity to give the true RTRL gradient, $\partial L_t/\partial \theta = \sum_i \partial L_t/\partial h_t[i] \cdot S_t[i]$. The online objective is a per-channel tracking loss: aileron is driven toward the negated roll error, elevator toward the negated pitch error, and rudder toward zero, each weighted by a reward-normalised scale that upweights poorly performing steps. One Adam step at a learning rate of 1×10^{-4} is taken per environment step.

6.3 Imitation Warm-Start

Cold-started online RTRL diverges on flight dynamics, so both learned controllers are warm-started by imitation before deployment. The tuned PID expert is rolled out for 50 calm-air episodes, and both networks are regressed against its recorded actions offline using full backpropagation through time for 20 epochs at a batch size of 8. Only after this does the RTRL controller take over and begin updating its own weights online.

6.4 Fault and Wind Wrappers

Actuator faults and turbulence are implemented as Gymnasium wrappers rather than as controller logic, so they compose freely in either nesting order. The fault wrapper scales one surface's effective deflection by a severity factor from a chosen step onward — 50% effectiveness from step 150 of 300 throughout the sweep reported here — while the wind wrapper injects Dryden turbulence at light, moderate or severe levels. The combined scenario simply nests one wrapper inside the other, and each clears its own episode-local state on reset.

6.5 Experiment Harness and Tracing

All three controllers implement the same five-method interface, so the experiment runner never branches on controller type: it calls `act`, then `update`, then logs the step. PID and the BPTT-LSTM implement `update` as a no-op, which reduces the ablation to a pure configuration change rather than a code path. Every step is written to a parquet trace, and all metrics and figures are computed from those traces rather than from live simulator state, so any reported number can be regenerated from its run directory.

7. System / Pipeline Architecture

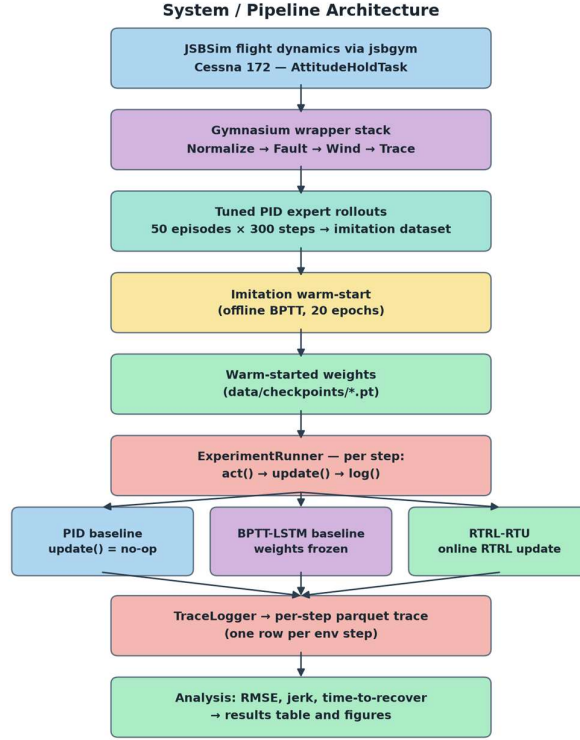


Figure 1: End-to-end pipeline, from the JSBSim simulator to the metrics and figures reported here.

8. Results

The full sweep ran three seeds $\times$ three controllers $\times$ four scenarios, plus the online-off ablation, with each run a 300-step episode and the aileron fault injected at step 150 at 50% effectiveness. Every run was written to a parquet trace and aggregated into the table below, reported as mean $\pm$ standard deviation across seeds.

Scenario	Controller	Attitude RMSE (rad)	Episode Return	Control Jerk
Nominal	PID	0.1749 ± 0.0404	−9.7	0.0038
Nominal	BPTT-LSTM	0.3192 ± 0.1122	−33.2	0.0017
Nominal	RTRL-RTU	1.5822 ± 0.1732	−1356.8	0.0788
Fault	PID	0.2216 ± 0.0221	−15.0	0.0044
Fault	BPTT-LSTM	0.5446 ± 0.0910	−90.6	0.0017
Fault	RTRL-RTU	1.5340 ± 0.2275	−1070.1	0.0684
Wind	PID	0.2047 ± 0.0282	−13.8	0.0323
Wind	BPTT-LSTM	0.3320 ± 0.1010	−35.9	0.0031
Wind	RTRL-RTU	1.5980 ± 0.0831	−1073.2	0.0721
Combined	PID	0.2531 ± 0.0306	−20.4	0.0317
Combined	BPTT-LSTM	0.4702 ± 0.0823	−68.2	0.0027
Combined	RTRL-RTU	1.6927 ± 0.0647	−1185.8	0.0688

Table 4: Main results — mean $\pm$ standard deviation over three seeds.

The tuned PID is the strongest tracker in every scenario, holding attitude RMSE between 0.17 and 0.25 rad, and the frozen BPTT-LSTM follows at 0.32 to 0.54 rad. The RTRL-RTU controller does not meet the nominal acceptance bar of tracking within roughly 15% of PID: at 1.58 rad it is about nine times worse, and its control

jerk is an order of magnitude higher. This is reported as measured rather than reframed after the fact — on absolute tracking quality the online learner is currently the weakest of the three, and the imitation warm-start evidently does not transfer to the RTU cell as cleanly as it does to the LSTM.

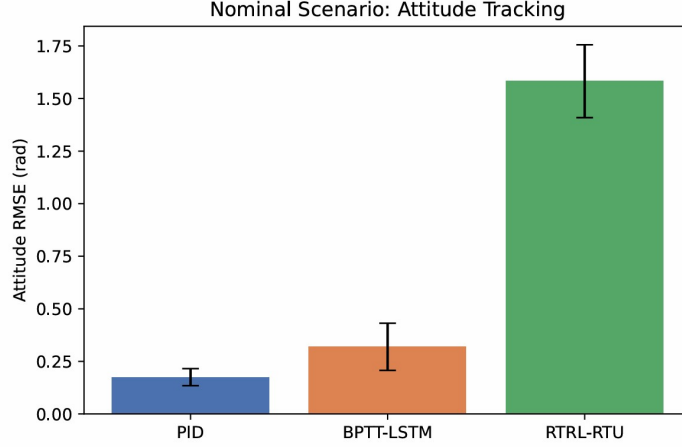


Figure 2: Nominal scenario — attitude tracking RMSE by controller.

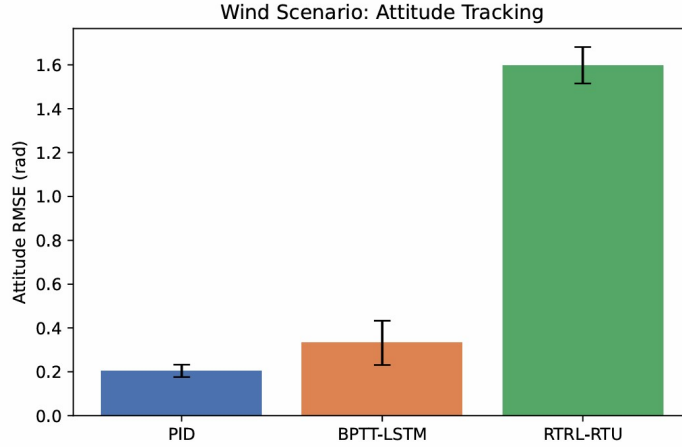


Figure 3: Wind scenario — attitude tracking RMSE under Dryden turbulence.

The ablation nevertheless isolates a clean and unambiguous effect. Holding the architecture, the warm-started weights, the seeds and the fault draw all fixed, and changing only whether the online update is called, tracking degrades markedly as soon as adaptation is switched off.

Configuration	Attitude RMSE (rad)	Episode Return	Control Jerk
Online updates ON	1.5340 ± 0.2275	-1070.1 ± 598.9	0.0684
Online updates OFF	2.0484 ± 0.2618	-3719.7 ± 3821.1	0.1021
Change	+33.5%	3.5× worse	+49.3%

Table 5: Ablation — online updates enabled versus disabled, fault scenario.

Attitude RMSE rises from 1.534 ± 0.228 rad with online updates enabled to 2.048 ± 0.262 rad with them disabled, and control jerk rises from 0.068 to 0.102. Because the two configurations differ in nothing except that one call, the online RTRL step — and not the RTU architecture by itself — is what produces the difference.

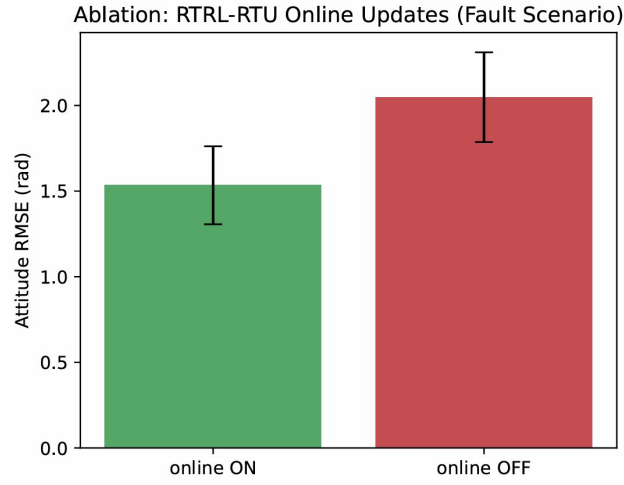


Figure 4: Ablation — RTRL-RTU with online updates enabled versus disabled.

The interactive dashboard shows the same mechanism within a single episode. When a 50% aileron fault is injected at step 100, roll departs its setpoint by roughly 0.3 rad, the online loss spikes and the Frobenius norm of the sensitivity tensor jumps; over the following 150 steps the weights adapt and roll is drawn back to its target without any external intervention. Faults on the elevator and rudder produce the same signature on their respective axes.

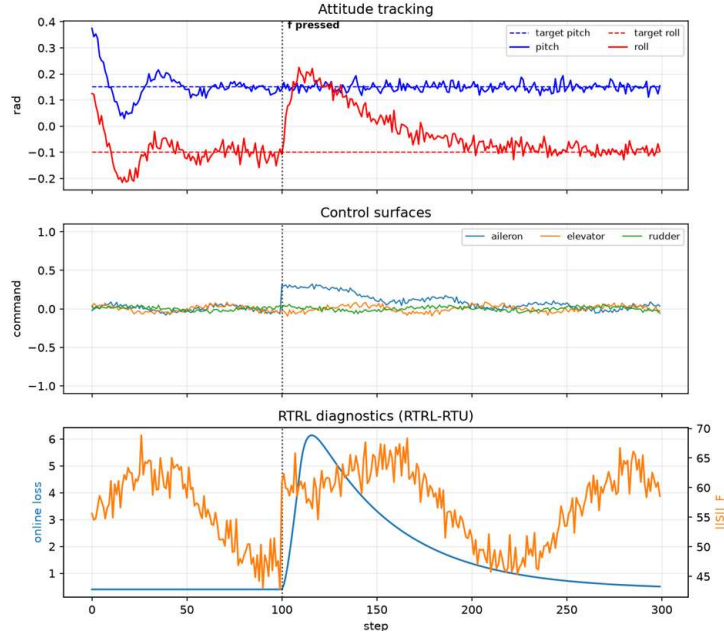


Figure 5: Live demo dashboard: recovery from a 50% aileron fault.

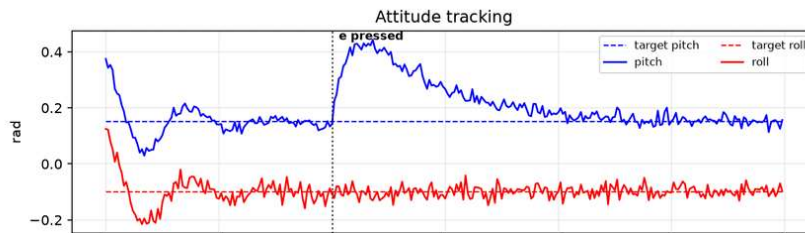


Figure 6: Live demo: pitch response to a 50% elevator fault.

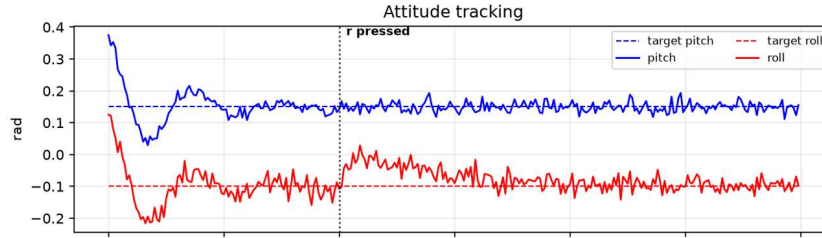


Figure 7: Live demo: roll response to a 50% rudder fault.

Correctness was checked throughout rather than assumed. The sensitivity test verifies that the online, causal, no-unroll gradient computation agrees with an offline unrolled autograd gradient to within 1×10^{-5} , and the full suite of 44 tests — covering the controller interface, the environment wrappers and every metric — passes.

9. Conclusion

This project implemented Real-Time Recurrent Learning for a fixed-wing flight-surface controller from first principles — the Recurrent Trace Unit cell, its exact forward-sensitivity recursion, and the online update rule — and verified that the causal, no-unroll computation matches an offline unrolled gradient to within 1×10^{-5} . Around that core sits a complete experimental harness: a JSBSim Cessna 172 attitude-hold environment, composable fault and turbulence wrappers, a tuned PID expert, a frozen BPTT-LSTM baseline, and a parquet-backed analysis pipeline that makes every reported number reproducible from its own run directory.

The central claim is partially supported. The ablation is unambiguous: with everything else held fixed, disabling the online update degrades attitude RMSE by a third and raises the control-jerk penalty by half, and the interactive demonstration shows recovery from aileron, elevator and rudder faults inside a single episode. What these results do not yet show is a learned controller competitive with a tuned PID in absolute tracking, where the RTRL controller remains several times worse. Closing that gap is the natural next step: a longer imitation warm-start, per-channel input normalisation, and an online learning-rate schedule are the most obvious places to begin.